

FastAPI CRUD Q1

1. What is CRUD in FastAPI?

CRUD stands for:

- **C** → Create
- **R** → Read
- **U** → Update
- **D** → Delete

In FastAPI, CRUD operations are used to manage data through API endpoints.

Example:

- `POST` → Create data
- `GET` → Read data
- `PUT/PATCH` → Update data
- `DELETE` → Delete data

2. Why do we use the POST method for inserting data?

The `POST` method is used to send new data to the server and create a new resource.

Example:

```
@app.post("/students/")
def add_student(student: Student):
    students.append(student)
```

It is mainly used because:

- It securely sends data in the request body.
- It is designed for creating new records.

3. Difference between PUT and PATCH

PUT	PATCH
Updates the entire record	Updates only specific fields
Requires full data	Requires partial data
Replaces old data completely	Modifies existing data

Example:

PUT -> update full student details
PATCH -> update only student age

4. What is the path parameter?

A path parameter is a value passed inside the URL path.

Example:

```
@app.get("/students/{student_id}")
def get_student(student_id: int):
    return {"id": student_id}
```

Here, `student_id` is the path parameter.

URL Example:

`/students/101`

5. What is the request body in FastAPI?

The request body contains data sent by the client to the API.

FastAPI usually uses JSON data in the request body.

Example:

```
class Student(BaseModel):
    name: str
    age: int

@app.post("/students/")
def create_student(student: Student):
    return student
```

Here, the student details are sent in the request body.

6. Why do we use Pydantic models?

Pydantic models are used for:

- Data validation
- Type checking
- Automatic data conversion
- Better documentation

Example:

```
class Student(BaseModel):  
    id: int  
    name: str
```

If wrong data types are sent, FastAPI automatically shows an error.

7. What status code is returned for successful deletion?

Usually:

200 OK

or

204 No Content

204 is commonly used when nothing is returned after deletion.

Example:

```
@app.delete("/students/{id}", status_code=204)
```

8. How can we check whether a student exists before updating?

We first search for the student ID in the database or list.

Example:

```
@app.put("/students/{id}")  
def update_student(id: int, student: Student):  
    if id not in students:  
        return {"message": "Student not found"}  
  
    students[id] = student  
    return {"message": "Updated successfully"}
```

This prevents updating non-existing records.

9. What happens if duplicate IDs are inserted?

Duplicate IDs can:

- Cause data conflicts
- Overwrite existing records
- Create database errors

To avoid this, we check whether the ID already exists before insertion.

Example:

```
if student.id in students:  
    return {"message": "ID already exists"}
```

10. How does FastAPI automatically generate Swagger UI?

FastAPI automatically generates Swagger UI using:

- Python type hints
- Pydantic models
- API route definitions

Swagger documentation is available at:

`/docs`

ReDoc documentation:

`/redoc`

FastAPI internally uses the OpenAPI standard to create interactive API documentation automatically.